

Areas for Improvement: Algorithm and Computational Methods

Solver Limitations

The current implementation relies on the `javascript-lp-solver` library, a lightweight JavaScript implementation of the Simplex method. While this library is convenient for browser-based deployment and performs well for our relatively small problem (five decision variables and six constraints), it has several notable limitations. First, it solves only the continuous linear programming relaxation — it has no built-in support for integer programming. As a result, we implemented our own integer search by iterating downward from the LP optimum and testing each candidate for feasibility. This approach works reliably for our model because the number of decision variables is small and the integrality gap (the difference between the continuous and integer optima) is typically just a few patients. However, for larger or more complex hospital models with dozens of resource types, this brute-force enumeration would become computationally impractical.

A more robust approach would be to use a dedicated Mixed-Integer Programming (MIP) solver such as GLPK, CBC, or Gurobi. These solvers use sophisticated techniques — branch-and-bound, cutting planes, and presolve reductions — to efficiently navigate the integer solution space without exhaustively checking every candidate. During development, we initially experimented with `glpk.js` (a WebAssembly port of the GNU Linear Programming Kit), but encountered compatibility issues with our server-side rendering framework. Adopting a mature MIP solver would not only improve performance for larger models but also provide exact optimality guarantees with formal dual bounds, rather than relying on our custom downward search.

Additionally, the current solver assumes all relationships are strictly linear. In reality, hospitals often experience economies of scale (bulk purchasing discounts on equipment, for example) or diseconomies of scale (staff burnout at high patient-to-staff ratios leading to diminishing returns). Incorporating such nonlinearities would require moving beyond linear programming

entirely, toward nonlinear programming (NLP) or piecewise-linear approximations within the LP framework.

Integer Rounding Strategy

Our integer solution search proceeds by starting at the floor of the continuous optimum and decrementing the patient count by one until a feasible allocation is found. While this guarantees optimality for our specific model structure — because all resources scale monotonically with patient count — it is not a general-purpose integer programming method. In particular, it assumes that the optimal integer solution is the one with the highest patient count where all resource requirements can be met within budget and space. This assumption holds because our model has a single objective (maximise patients) and all constraints tighten as patients increase.

However, if the model were extended to include more complex trade-offs — for example, minimising cost while meeting a minimum patient target, or balancing multiple objectives such as patient throughput and staff wellbeing — the simple downward search would no longer suffice. In such cases, a proper branch-and-bound algorithm or a multi-objective optimisation framework would be necessary.

A further limitation is that our rounding applies the ceiling function to each resource independently. While this is correct for feasibility (you cannot hire a fraction of a doctor), it does not explore whether alternative integer combinations might yield a higher patient count. For instance, rounding nurses up by one might free enough budget to also add an extra bed, enabling one more patient than the straightforward ceiling approach would suggest. A true MIP solver would explore such trade-offs automatically.

Sensitivity Analysis Methodology

The sensitivity analysis engine provides valuable insights but relies entirely on finite-difference approximations rather than analytical methods. For each shadow price estimate, the engine bumps a constraint by a small epsilon (1% of its current value), re-solves, and computes the ratio $\Delta P / \Delta \epsilon$. This numerical approach has two drawbacks. First, the choice of epsilon is somewhat arbitrary — too small and numerical noise dominates, too large and the linear

approximation breaks down. Second, it requires a full re-solve for each perturbation, which is computationally expensive.

A superior approach would be to extract dual variables (shadow prices) directly from the LP solver's output. The Simplex method naturally computes these as part of its solution process, so they are available at zero additional cost and are mathematically exact rather than approximate. The `javascript-lp-solver` library we use does not expose dual variables, which is why we resorted to numerical estimation. Switching to a solver that provides dual information (such as GLPK or any commercial solver) would yield exact shadow prices, reduced costs, and basis status for each constraint — all of which are standard outputs in operations research practice.

The parameter sweep methodology — varying one parameter at a time while holding all others fixed — is a standard one-at-a-time (OAT) sensitivity analysis. While intuitive and easy to interpret, OAT analysis does not capture interaction effects between parameters. For example, increasing the budget and simultaneously reducing the cost per doctor might produce a synergistic effect that is larger than the sum of their individual impacts. More advanced techniques such as Design of Experiments (DOE), Monte Carlo simulation, or Sobol indices could quantify these interactions. Monte Carlo simulation, in particular, would allow assigning probability distributions to uncertain parameters (rather than fixed $\pm 20\%$ ranges) and would produce a distribution of outcomes rather than point estimates, giving decision-makers a more complete picture of risk.

The heatmap currently explores only the Budget $\times$ Space interaction. While these are arguably the two most important macro-constraints, a more comprehensive analysis would generate heatmaps for other parameter pairs — for example, Cost per Doctor $\times$ Patients per Doctor, or Budget $\times$ Minimum Staffing. Automating the generation of all pairwise heatmaps would provide a fuller picture of the solution landscape.

Computational Performance

The sensitivity analysis engine runs approximately 1,500 solver iterations to produce its full suite of results. On modern hardware, this completes in under two seconds, which is acceptable for an interactive web application. However, the current implementation runs these iterations sequentially. If the model

were scaled up — more parameters, finer sweep granularity, or higher-dimensional heatmaps — performance could degrade noticeably.

Two improvements could address this. First, the solver iterations within each analysis (sweeps, tornado, heatmap) are independent of one another and could be parallelised using Web Workers or server-side threading. Second, many of the solver calls share similar inputs and could benefit from warm-starting — initialising the Simplex method from a nearby feasible basis rather than solving from scratch each time. Warm-starting is a standard feature in production-grade LP solvers and can reduce iteration counts by an order of magnitude for closely related problems.

Model Extensibility

The current algorithm is hard-coded for four resource types (doctors, nurses, equipment, beds) with two aggregate constraints (budget and space). A more extensible architecture would allow users to define arbitrary resource types and constraints through the interface, with the solver dynamically constructing the LP model from a general specification. This would enable the tool to be applied to a wider range of hospital configurations — for example, adding pharmacists, operating theatres, or specialised imaging equipment as additional decision variables, each with their own cost, space, and capacity parameters.

Furthermore, the model is static — it optimises for a single point in time with fixed parameters. A natural extension would be to introduce a time dimension, allowing the model to account for seasonal demand fluctuations, phased capital investments, or staff hiring timelines. This would transform the problem from a single-period LP into a multi-period optimisation problem, significantly increasing both its realism and its computational complexity.